Session 1 — Terminal & Git fundamentals

Git, Python Environments & Coding Best Practices

Guillaume Leveque

Session 1 of 4

Master of Data & AI — MSA-DATB07-03

Why this course (5 min)

The folder everybody has

```
analysis.py
analysis_v2.py
analysis_v2_final.py
analysis_v2_final_REAL.py
analysis_v2_final_REAL_bob.py
copy of analysis (1).py
```

What goes wrong:

- Which file is the right one? What changed between two of them, who changed it, and why?
- Two people edit the same file: one silently overwrites the other.
- "It works on my laptop" but not on yours: different Python, different packages.

What this course gives you

Track every change and go back in time. **Work as a team** on the same code without overwriting each other. **Share the same environment** and the same quality rules, so that everyone's code runs everywhere.

Four sessions, one project

#	topic	you leave able to ...
1	Terminal & Git basics	use the terminal; save versions of your work and go back
2	Branches & merges	work on several things in parallel, combine them, fix conflicts
3	GitHub & pull requests	share your work, review teammates' code, contribute
4	Environments & code quality	recreate the same Python setup on any machine; let tools check the code for you

One small project, `sales-analysis`, follows you through the four practicals: you build it, branch it, share it, then clean it up.

How you are assessed

One multiple-choice quiz (QCM) — 100 % of the grade, on the four sessions. The kinds of question:

- what does this command do? (each part of it)
- which command, or which order of commands, gives this result?
- read the state of a repository (a `git log`, a commit graph): what happened, what will happen next?

The TDs are **not graded**: nothing to send, nothing to show. They are your practice for the quiz:

- the "Check yourself" boxes in the slides,
- the questions marked ★ in the TDs,
- the "Read this repository" slide at the end of each session.

Terminal, shell, OS (7 min)

The terminal: talking to the computer in text

- When you double-click an icon, you ask the computer to *run a program*.
- Applications do this all the time behind the scenes: the *Run* button of VS Code runs `python analysis.py` for you.
- The terminal lets you give these orders yourself, by typing them.

Why bother?

- **precise**: you see exactly what runs
- **repeatable**: copy a command, put it in a script
- **everywhere**: servers have no mouse

```
python analysis.py      # what the Run button does: "python, please run analysis.py"
```

Terminal = the window where you type. **Shell** = the program inside it that reads and runs your commands (`bash`, `zsh`, PowerShell, `cmd`). **OS** = Windows, macOS, Linux.

One shell for everyone: bash

Windows comes with two shells, neither of them bash:

- `cmd.exe`: `dir`, `copy`, backslashes
- PowerShell: its commands look like bash (`ls`, `cd`) but many differ (`rm -rf`, `touch`, `export`)

macOS comes with `zsh`, which behaves like bash for everything in this course. Linux comes with `bash`.

Our choice

So that every command in this course exists in one version only, everybody uses a bash shell: **Git Bash** on Windows (it comes with Git for Windows) and **Terminal** on macOS, both usable inside VS Code. Data science tools are documented for this kind of shell.

What you will install at the start of the TD

	Windows	**macOS**
Git + a bash shell	*Git for Windows*: gives Git **and** Git Bash	comes with the Apple developer tools
Python 3.10 or newer	python.org, separate install	python.org, separate install
Editor	VS Code + Python extension	VS Code + Python extension

Pitfall — Git does not install Python

Git for Windows gives you Git and Git Bash, nothing else. Python is a separate program with its own installer. On Windows, tick *Add python.exe to PATH* in the Python installer.

Which shell am I in? Two pitfalls

```
echo $0      # echo prints what you give it; $0 = the name of the running program, here the
     shell
```

answer	meaning
contains `bash` (`bash`, `/usr/bin/bash`)	Git Bash: good
contains `zsh` (`zsh`, `-zsh`)	macOS Terminal: good
an empty line	PowerShell: open Git Bash instead
$0 printed as is	cmd: open Git Bash instead

Pitfall — python vs python3

On macOS the command is usually `python3`, not `python`. The slides write `python` to stay short. Session 4 removes the difference with virtual environments.

Navigating and handling files (15 min)

Where am I, what is here, go somewhere

```
pwd             # "print working directory": where am I?
ls              # "list": the files and folders here
ls -l           # -l = long format: size, date, permissions
ls -la          # -a = all, including hidden files (names starting with a dot)
cd data/raw     # "change directory": go into data, then raw
cd ..           # .. = the folder above
cd ~            # ~ = your home folder
cd -            # back to the previous folder
```

Options start with a dash and change what a command does: `ls` lists, `ls -l` lists with details. They can be combined: `-la` = `-l` + `-a`.

Paths: the address of a file

Absolute path: from the top of the disk.
`/Users/alice/Documents/courses/git` (macOS)
`/c/Users/alice/Documents/courses/git` (Git Bash)

Relative path: from where you are.
`data/raw/sales.csv`, `../other_project`

`.`	this folder
`..`	the folder above
`~`	your home folder
`/`	separator, and the top of the disk

- **Never type a full path**: type the first letters and press `Tab` (twice to see all the candidates); or type `cd` and a space, then drag a folder from the file explorer onto the terminal: its path is typed for you.
- **No spaces in names**: `cd my project` looks for a folder called `my`. Use underscores: `my_project`, `sales_2024.csv`. For a path you cannot rename, use quotes: `cd "OneDrive - Company"`.

Create, copy, move, delete

```
mkdir outputs             # "make directory": create a folder
mkdir -p data/raw/2024    # -p = create the whole chain of folders at once
touch notes.md            # create an empty file
cp sales.csv backup.csv   # "copy": cp <from> <to>
cp -r data data_backup    # -r = recursive: copy a folder and everything inside
mv notes.md README.md     # "move": also the way to rename
mv README.md docs/        # move into a folder
rm backup.csv             # "remove": delete a file
rm -r data_backup         # delete a folder and everything inside
```

Pitfall — there is no recycle bin

`rm` deletes immediately and for good. Read the path twice before pressing `Enter`. Git will give you a safety net for the files it tracks.

Look inside files

```
cat README.md           # print the whole file, then give the prompt back
less logs/server.log    # read page by page: Space = next page, /word = search, q = quit
head -n 5 sales.csv     # the first 5 lines (-n = number of lines)
tail -n 5 sales.csv     # the last 5 lines
wc -l sales.csv         # "word count", -l = count the lines
code analysis.py        # open the file in VS Code, to edit it
```

- `cat` and `less` only *read*. To *modify* a file, we open it in VS Code with `code`.
- First reflex on a new dataset: `head -n 3 file.csv` shows the column names and the format.

Keyboard habits

`Tab`	complete a file, folder or command name
`Up` / `Down`	scroll through the commands you already typed
`Ctrl+R`, type a few letters	search in that history; `Ctrl+R` again = an older match
then `Enter` / an arrow key	run it / put it on the line to edit it
`Ctrl+C`	**stop** the running command (it does not copy!)
`Ctrl+L` or `clear`	clean the screen
`Ctrl+Insert` / `Shift+Insert`	copy / paste (Windows: Git Bash and VS Code)
`Cmd+C` / `Cmd+V`	copy / paste (macOS)
`code .`	open the current folder in VS Code

All the commands containing "git" that you typed: `history | grep git`; then `!42` runs number 42 again.

Stuck? How to get out

what you see	what it is	to get out
`:` or `(END)` at the bottom	a long list: `less`, `git log`, `git show`, `git diff`	`q`
lines of `~` on a black screen, maybe `-- INSERT --`	the *vim* editor, opened by Git for a message	`Esc`, then `:wq` `Enter` (save and quit) or `:q!` `Enter` (quit, no save)
`^X Exit` at the bottom	the *nano* editor	`Ctrl+X`
a command that never ends	a running program (`ping` on macOS…)	`Ctrl+C`
the prompt `>>>`	Python, interactive	`exit()`
a prompt `>` after your command	a quote that was never closed	`Ctrl+C`
“Waiting for your editor to close the file”	Git waits for VS Code	save and close the tab

Demo folder: shell-lab

The TD1 repository contains a practice folder. I use it now for the live demo; you will explore it yourself during the TD.

```
shell-lab/
|-- README.txt
|-- data/
|   |-- raw/      sales_2024_q1.csv ... q4.csv, README.txt
|   `-- processed/
|-- docs/
|   |-- notes/    meeting_2024-01-15.md, meeting_2024-02-12.md
|   |-- archive/  2019/old_report.txt, 2020/old_report.txt
|   `-- glossary.md
|-- logs/         server.log (300 lines), app.log
`-- scripts/      clean.py, report.sh
```

In the TD you will find out: how many sales the first quarter contains, which file mentions `PENGUIN`, and what is hidden in this folder.

Demo: exploring shell-lab

```
cd shell-lab                          # go into the practice folder
pwd                                   # check where I am
ls                                    # what is here?
ls data/raw                           # what is inside data/raw? (no need to go there)
head -n 3 data/raw/sales_2024_q2.csv  # the first 3 lines: column names + 2 sales
wc -l data/raw/sales_2024_q2.csv      # 56 lines = 1 line of column names + 55 sales
ls data/processed                     # looks empty...
ls -la data/processed                 # ... but -a shows a hidden file: .gitkeep
cat data/raw/README.txt               # read a short text file
```

- Names starting with a dot are hidden: only `ls -a` (or `-la`) shows them.
- `.gitkeep`: Git saves files, not folders, so an empty folder would not be kept. An empty file named `.gitkeep` (a convention, any name would work) makes Git keep the folder.

Redirections, search, environment (11 min)

Send the output to a file: > and >>

A command normally prints its result on the screen. > and >> send it into a file instead.

- > **replaces**: the file is emptied, then written (created if it does not exist).
- >> **adds**: the text goes at the end of the file, the old content is kept.

```
echo "a" > f.txt        # f.txt contains: a
cat f.txt
echo "b" > f.txt        # f.txt contains: b          (a was erased)
cat f.txt
echo "c" >> f.txt       # f.txt contains: b then c   (c was added at the end)
cat f.txt
```

echo simply prints the text you give it. Real use: `ls -la > listing.txt` saves a listing in a file.

Chain two commands: the pipe |

The pipe | gives the output of the first command to the second one, instead of printing it.

```
grep ERROR logs/server.log | wc -l       # the error lines, then count them
grep ERROR logs/server.log | head -n 3   # the error lines, keep only the first 3
ls data/raw | wc -l                      # list the files, then count them
```

- Read it from left to right: "take the error lines, *then* count them".
- Small commands that each do one thing, chained together: that is the heart of the shell.

Tip — one step at a time

Run the first command alone, look at the result, then add | and the next one.

Search inside files (grep) and for files (find)

```
grep ERROR logs/server.log          # the lines that contain ERROR
grep -c ERROR logs/server.log       # -c = only count them
grep -i error logs/server.log       # -i = ignore upper/lower case
grep -r PENGUIN .                   # -r = in every file of every sub-folder of . (here)
grep -rl PENGUIN .                  # -l = only the names of the files
find . -name "*.md"                 # files whose NAME ends in .md, from here downwards
find . -type d                      # -type d = only folders ("directories")
```

- `grep` searches *inside* files, `find` searches file *names*. Both start from the folder you give them: `.` means "here".
- Wildcards: `*` = any text. `grep Widget data/raw/*.csv` searches every CSV of `data/raw`.

Environment variables and PATH

The shell keeps named values, called *variables*. `$NAME` reads one.

```
echo $HOME                  # your home folder
echo $PATH                  # the list of folders where the shell looks for programs
which python                # which python will run when I type "python"?
export PROJECT=sales        # create a variable for this terminal window
echo $PROJECT               # sales
```

- When you type `python`, the shell looks in each folder of `PATH`, in order, and runs the *first* `python` it finds. This explains most "wrong Python version" problems (session 4).
- A variable created with `export` disappears when you close the window.

Shortcuts that stay: aliases

```
alias ll='ls -la'          # shortcut: typing ll now runs ls -la
```

An alias typed in the terminal disappears when you close the window. To keep it, write it in the file the shell reads at every start: `~/.bashrc` (Git Bash), `~/.zshrc` (macOS).

```
echo "alias ll='ls -la'" >> ~/.bashrc     # >> adds the line at the end of the file
tail -n 3 ~/.bashrc                       # check
source ~/.bashrc                          # apply it now (or open a new terminal)
```

- **Never > here**: it would empty the file and erase your whole configuration.
- The double quotes keep the single quotes of the alias inside the line. Or open the file with `code ~/.bashrc`, add the line, save.
- The first time, Git Bash may warn "Found ~/.bashrc but no ~/.bash_profile": harmless, it creates the missing file.

Never write a secret in your files

- A *secret* is anything that gives access: a password, or an *API key*.
- An *API key* is a password that a program uses to access an online service: a weather service, a database, an AI model.
- A secret written in a file can end up in Git, then on GitHub, visible to everyone.

We will see how to keep secrets out of Git with `.gitignore` later today.

Libraries with pip (3 min)

Reminder: install a library with pip

A *library* is code already written by others: to draw a chart, you do not draw it yourself. `pip` downloads and installs libraries.

```
python -m pip install plotly   # install
python -m pip list             # what is
    installed
python -m pip show plotly      # version,
    location
```

`python -m pip` = the pip of *this* python: you install where you run. macOS: `python3 -m pip`.

```
# chart.py: sales per quarter, from shell-lab
import plotly.graph_objects as go

quarters = ["Q1", "Q2", "Q3", "Q4"]
sales = [40, 55, 30, 65]

fig = go.Figure(go.Bar(x=quarters, y=sales))
fig.update_layout(title="Sales per quarter,
    2024")
fig.show()   # opens the chart in the browser
```

`python chart.py`

Today everything is installed in one place shared by all your projects. Session 4 shows why this becomes a problem, and how to keep each project apart.

Network and system survival kit (10 min)

Why a survival kit

- “The website does not answer”: is it down, or is it my connection? `ping`
- “Download the data from a URL”, “what does this API answer?” `curl`
- “My notebook is frozen”: which program, how to stop it? `ps`, `kill`
- “Permission denied” on a script you just wrote. `ls -l`, `chmod`

More in the appendix and on the cheat-sheet: the route to a server, names and addresses, ports, disk space, archives.

Is it reachable? ping

`ping` sends a few small test messages to a machine and measures how long the answer takes. Nothing of yours is sent.

```
ping github.com            # Windows: sends 4 test messages, then stops
ping -c 4 github.com       # macOS: -c 4 = "count", send 4 (otherwise it never stops: Ctrl+C)
```

```
64 bytes from 140.82.121.4: icmp_seq=1 ttl=52 time=18.3 ms
...
4 packets transmitted, 4 received, 0% packet loss
```

- `time`: the round trip in milliseconds. `0% packet loss`: every message came back.
- 100 % loss while the website opens in your browser: the network blocks `ping`, the server is fine.

Talk to a website: curl

`curl` fetches a web address from the terminal, like a browser that shows the raw text.

```
curl -I https://github.com                # -I = only the headers (status, server...)
curl -o sales.csv https://example.com/sales.csv   # -o = save into a file instead of
    printing
curl -L -o data.zip https://example.com/data      # -L = follow redirections
```

The first line of `curl -I` is the *status code*:

200	OK
301 / 302	moved elsewhere (`-L` follows it)
403 / 404	forbidden / not found
500 and above	the server has a problem, not you

Ask a service for data: an API and JSON

An *API* is a web address that answers with data instead of a page, often in *JSON*, a text format that looks like a Python dictionary.

```
curl -s "https://api.open-meteo.com/v1/forecast?latitude=48.85&longitude=2.35&current=
    temperature_2m" | python -m json.tool
```

piece	meaning
`curl -s "..."`	fetch the data; `-s` = silent, no progress bar
the quotes	needed: the address contains `&`, which means something else for the shell
`| python -m json.tool`	give the answer to Python's JSON tool, which indents it

The answer contains `"current": {"temperature_2m": 14.2}`: the temperature in Paris now.

Windows error `CRYPT_E_NO_REVOCATION_CHECK`: add `--ssl-no-revoke` (see the TD).

Processes: what is running, how to stop it

Every running program is a *process* with a number, its *PID*.

```
ps                          # my processes in this terminal (macOS: ps aux = all of them)
ps aux | grep python        # macOS/Linux: the python processes and their PID
kill 12345                  # ask process 12345 to stop
kill -9 12345               # force it (last resort: it cannot save anything)
```

- A command still running in your terminal: `Ctrl+C`.
- On Windows, the Task Manager (`Ctrl+Shift+Esc`) shows every process with its PID.

Permission denied: read ls -l, then chmod +x

```
$ ls -l scripts/report.sh
-rw-r--r--  1 alice  staff  210 Mar 12 10:02 scripts/report.sh     <- before
$ chmod +x scripts/report.sh                                      <- "change mode": add x
$ ls -l scripts/report.sh
-rwxr-xr-x  1 alice  staff  210 Mar 12 10:02 scripts/report.sh     <- after
$ ./scripts/report.sh                                             <- run it
```

	type	**owner** (you)	**group**	**others**
before	`-rw-r--r--`	`-`	`rw-`	`r--`	`r--`
after	`-rwxr-xr-x`	`-`	`rwx`	`r-x`	`r-x`

- Each group is always 3 characters, in the order `r` (read), `w` (write), `x` (execute); `-` = not allowed. `chmod +x` adds the `x` in every group.
- Opening the file (VS Code, double-click) shows its content; running it is `./scripts/report.sh` (`./` = "the file here"). `bash scripts/report.sh` also runs it, even without `x`.
- Windows: Git Bash already shows the `x` for a `.sh` script. macOS, Linux, servers: `chmod +x` is needed.

Why Git: the problem, the model (9 min)

A familiar story

Alice and Bob work on `analysis.py`, exchanged by e-mail or on a shared drive.

- Monday: Alice adds a monthly total, sends `analysis_v2.py`.
- Tuesday: Bob fixes a bug in his own copy of the *first* version, sends `analysis_v2_bob.py`.
- Wednesday: the monthly totals are wrong, and nobody knows since when.

Questions nobody can answer:

1. Which file is the current version?
2. What exactly changed since last week, who changed it, and why?
3. How do we get back the version that worked on Monday?
4. How do we combine Alice's and Bob's changes without losing one of them?

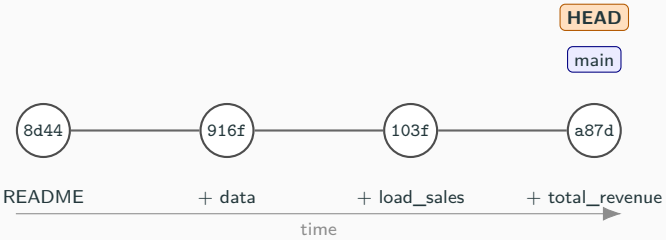
What Git gives you

need	Git's answer
one current version	one file name, one folder; the history is kept apart
what changed, who, why	every saved version has an author, a date and a message
go back	any saved version can be brought back
combine work	branches and merges (session 2)
share	GitHub (session 3)

Git vs GitHub

Git is a program on your computer; it works offline. **GitHub** is a website that stores Git projects online and adds team features. Today: Git only.

A chain of snapshots



- A **commit** is a saved version of the whole project: a snapshot, with a message, an author and a date. Think of a save point in a video game.
- Each commit has an identifier (`8d44929...`), called its *hash*. The first 4 to 7 characters are enough to name it.
- `main` is the name of the line of work. `HEAD` means "you are here".

The three areas



- You edit files freely. Nothing is saved in Git until you ask.
- `git add` puts a change in the *staging area*: "this goes into my next save".
- `git commit` saves everything that is staged as one commit.
- Two steps on purpose: you choose what belongs together in one save point.

The .git folder

```
git init            # turn the current folder into a Git repository
ls -la              # a hidden folder .git appeared
```

- The whole history lives in `.git`. Never edit it by hand.
- Delete `.git` and your files stay, but the history is gone.
- One repository = one project folder. Check where you are (`pwd`) before `git init`: never in your home folder.

Pitfall — git init in the wrong place

`git status` lists hundreds of files from all over your disk: you ran `git init` in a folder that is too high. Go to that folder, delete its `.git` (`rm -rf .git`), and run `git init` in the project folder.

The core loop (15 min)

Once per computer: tell Git who you are

```
git config --global user.name  "Alice Martin"
git config --global user.email "alice.martin@student.example.edu"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
```

`--global`	for every repository on this computer
`user.name`, `user.email`	written on every commit you make; use the e-mail of your future GitHub account
`init.defaultBranch main`	name of the first line of work in a new repository. Old Git versions say `master`; we all use `main`
`core.editor "code --wait"`	when Git needs you to write a text (a message), it opens VS Code and waits until you close the tab

The loop you will run a thousand times

```
mkdir my_sales && cd my_sales            # && = then, if the first one worked
git init
code analysis.py                         # write a few lines, save
git status                               # "untracked": Git does not know it yet
git add analysis.py                      # stage it
git commit -m "Add load_sales function" # -m = the message
git log --oneline                        # the history
```

A good message: short, starts with a verb, says *what* the commit does: `Add monthly_revenue function`, `Fix crash on empty CSV`. Useless: `fix`, `stuff`, `final version`. One change, one commit.

Pitfall — "LF will be replaced by CRLF"

On Windows Git prints this when you add a file. It is about invisible end-of-line characters and is harmless: ignore it.

Reading git status

```
On branch main
Changes to be committed:            <- staged: will be in the next commit
        new file:   data/sales.csv
Changes not staged for commit:      <- modified, but not staged yet
        modified:   analysis.py
Untracked files:                    <- files Git does not follow at all
        notes.txt
```

- The same file can appear twice: one version staged, newer edits not staged.
- "nothing to commit, working tree clean": everything is saved. The state you want before closing your laptop.

Reading the history: log and show

```
git log                  # every commit: identifier, author, date, message
git log --oneline        # one line per commit: short identifier + message, newest on top
git log -n 3             # -n 3 = only the last three
git log --stat           # with the list of files changed in each commit
git show 8316eef         # one commit in detail: who, when, message, and the changes
git show HEAD~1          # HEAD~1 = the commit before the current one
```

- `git log` shows the commits of the line of work you are on, not your unsaved edits (that is `git status`).
- `HEAD` is the commit you are on, `HEAD~1` the one before, `HEAD~3` three steps back.
- A long list waits at the bottom of the screen (`:`): `Space` for the next page, `q` to quit.

What changed: git diff

```
git diff                    # my edits that are NOT staged yet
git diff --staged           # what IS staged: exactly what the next commit will contain
git diff HEAD~3 HEAD        # between two commits
```

```
--- a/analysis.py
+++ b/analysis.py
@@ -12,3 +12,5 @@ def total_revenue(rows):
+    if not rows:
+        return 0.0
     return sum(float(r["quantity"]) * float(r["unit_price"]) for r in rows)
```

Lines starting with + were added, with − removed; `@@ -12,3 +12,5 @@` says where in the file.
Habit: read `git diff --staged` before every commit.

.gitignore and undoing (12 min)

.gitignore: what never goes into Git

```
# .gitignore : a text file at the root of the repository, one pattern per line
__pycache__/
.venv/
outputs/
*.png
.env
```

Ignore three kinds of files:

- **generated**: caches, outputs, environments (`__pycache__/`, `outputs/`, `.venv/`)
- **big**: raw data beyond a few MB
- **secret**: passwords and API keys, usually kept in a file called `.env`

Pitfall — too late for files already saved

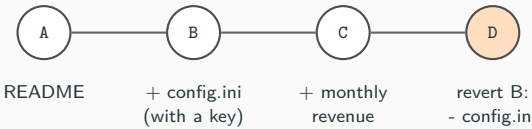
`.gitignore` only affects files Git does not follow yet. A file already committed stays followed until you run `git rm --cached file` and commit.

Undo, part 1: changes not committed yet

```
git restore --staged analysis.py   # unstage: the edit stays in the file, it just leaves the
      staging area
git restore analysis.py            # throw away my edits: the file goes back to the last
     commit
```

- `--staged`: nothing is lost.
- Without `--staged`: your uncommitted edits to that file are gone for good. Git cannot bring back what was never committed.

Undo, part 2: a bad commit, use git revert




```
git revert da319b1          # B: create a NEW commit D that does the opposite of B
```

- Revert does **not** go back in time: D removes only what B added. The work of C (`monthly_revenue`) stays.
- Nothing is erased, the history only grows: always safe, even when teammates already have B (session 3).
- VS Code opens with a message `Revert "..."`: keep it, save, close the tab.
- If a later commit changed the same lines as B, or lines right next to them, Git stops with a *conflict* (session 2); `git revert --abort` gets back to the state before.

git reset: the exception



after `git reset --hard A`:
B *and* C are gone,
with the work of C

```
git reset --soft HEAD~1    # undo my last commit, keep its changes staged, to redo it
    properly
```

- `git reset` moves the line of work backwards: the commits after that point disappear.
- Only acceptable for a commit you *just* made, that exists only on your laptop (forgot a file, wrong message). `--soft` keeps all your work.
- `git reset --hard` also throws away every uncommitted change: avoid it.

`git reflog` lists every place `HEAD` has been: a commit removed by mistake can be found there and brought back.

Safe or not: summary

command	effect	risk
`git status`, `log`, `show`, `diff`	read only	**safe**
`git add`, `git commit`	save a new version	**safe**
`git restore --staged f`	unstage, keep the edit	**safe**
`git revert <id>`	new commit cancelling one commit	**safe** (the default way to undo)
`git reset --soft HEAD~1`	remove my last commit, keep the work	**rewrites local history** (exception only)
`git restore f`	throw away uncommitted edits to f	**destroys work**
`git reset --hard`	remove commits and edits	**destroys work** (avoid)
`rm -rf .git`	delete the whole history	**destroys work**

Keep this table: it grows a little every session.

Wrap-up (3 min)

Read this repository

```
$ git log --oneline
79ee015 (HEAD -> main) Add monthly_revenue function
da319b1 Add config for the reporting API
f60a1f8 Add shell lab material for TD1
$ git show --stat da319b1 | tail -n 2
 project/config.ini | 3 +++
```

Check yourself — the kind of question of the quiz

1. Alice notices that `config.ini` contains an API key. Which command removes the file from the current version without erasing history?
2. After that command, what does the first line of `git log --oneline` look like?
3. The key is still visible in the old commit. What should Alice do with the key itself?

Read this repository: the answers

1. `git revert da319b1`: a new commit that removes `config.ini`. Nothing is erased, and the later commit `monthly_revenue` stays.
2. A new line on top: `xxxxxxx (HEAD -> main) Revert "Add config for the reporting API"` (the identifier differs).
3. Consider the key public: cancel it where it was created and make a new one. The old commit still contains it, and anyone with a copy of the repository can read it.

Take-aways and TD1

Remember

- one shell for everyone: Git Bash on Windows
- `pwd`, `ls`, `cd`, `Tab`, `q` and `Ctrl+C` to get out
- `>` `>>` `|`, `grep`, `find`
- edit → `git add` → `git commit`
- `git status` and `git diff` before, `git log` after
- undo a commit: `git revert`

TD1 (1 h 30)

1. install Git, Python, VS Code; configure Git (20 min)
2. explore `shell-lab` with the terminal (30 min)
3. your first repository, a history, undoing (35 min)
4. checklist (5 min)

Not graded. Questions marked ★ are the kind asked in the quiz.

`https://github.com/msa-datb07-git`

Next session: branches, merges and your first conflict.

Appendix: more survival commands

Appendix — Where does it stop? tracert / traceroute

```
tracert -d github.com      # Windows   (-d = show numbers, skip name lookups: faster)
traceroute -n github.com   # macOS     (-n = the same option, other name)
```

```
 1     1 ms    192.168.1.1        <- my box / router
 2    12 ms    10.20.0.1          <- my internet provider
 3     *       *       *          <- a step that does not answer (normal)
 9    19 ms    140.82.121.4       <- destination reached
```

- One line per *hop*, each machine your message goes through on its way.
- `* * *`: that machine does not answer this kind of test. It is not a failure as long as later lines appear.
- If everything stops after line 1, the problem is at home or on campus, not on the server.

Appendix — Names and addresses: nslookup, ipconfig

On the internet, machines are found by a number, the *IP address*. The *DNS* works like a phone book: it turns a name into that number.

```
nslookup github.com          # name -> IP address, and which DNS server answered
ipconfig                     # Windows: my own IP address
ipconfig getifaddr en0       # macOS: my own IP address on Wi-Fi
```

- `nslookup` fails but `ping 140.82.121.4` works: the phone book (DNS) is broken, not the network.
- Your address starts with `192.168.` or `10.`: a private address inside your home or campus network.

Appendix — Ports: who is using 8888?

One machine runs many services. Each one listens on a numbered *port*: Jupyter uses 8888, websites 443.

```
netstat -ano | findstr 8888    # Windows
lsof -i :8888                  # macOS
```

piece	meaning
`netstat -ano`	list the connections: `-a` all, `-n` numbers, `-o` with the program number (PID)
`| findstr 8888`	keep the lines with 8888 (`findstr` is the Windows `grep`)
`lsof -i :8888`	macOS: "list open files" using network port 8888

"Address already in use": an old Jupyter still holds the port. The last column gives its PID; stop it with `kill`.

Appendix — Disk: what takes the space?

```
df -h                    # "disk free": space left on each disk, -h = readable (G, M)
du -sh data              # "disk usage" of one folder: -s = total only, -h = readable
du -sh * | sort -h       # size of everything here, smallest first, biggest last
```

- `*` = every file and folder here; `sort -h` sorts sizes written as `12K`, `3.4M`, `1.2G`.
- Usual suspects in a data project: raw data, generated outputs, `.venv` (session 4).

Appendix — Pack a folder into one file: tar, zip

```
tar czf logs.tar.gz logs/    # create logs.tar.gz from the logs folder
tar tzf logs.tar.gz          # list what is inside, without extracting
tar xzf logs.tar.gz          # extract
```

The letters after `tar`: `c` = create, `t` = list, `x` = extract, `z` = compress (gzip), `f` = the file name follows.

`zip -r logs.zip logs/` and `unzip logs.zip` do the same with `.zip` files on macOS. Git Bash has `tar` but not always `zip`.